EIE2111
Pointers, Command-Line Processing and Linked List

Dr. Lawrence Cheung

Outline

- Pointers
  - Declarations and operations
  - The similarities and differences between pointers and references.
  - To use pointers to pass arguments to functions by reference.
  - To use pointer-based C-style strings.
  - The close relationships among pointers, arrays and C-style strings.

Outline

- Command-Line Processing
- Linked List
  - Introduction
  - Linked List
  - Stack
  - Queue
  - Tree

8.1 Introduction

- Pointers
  - Powerful, but difficult to master
  - Can be used to perform pass-by-reference.
  - Can be used to create and manipulate dynamic data structures.
  - Close relationship with arrays and strings
    - `char *` pointer-based strings

8.2 Pointer Variable Declarations and Initialization

- Pointer variables
  - Contain memory addresses as values.
    - Normally, variable contains specific value (direct reference).
    - Pointers contain address of variable that has specific value (indirect reference).
- Indirection
  - Referencing value through pointer

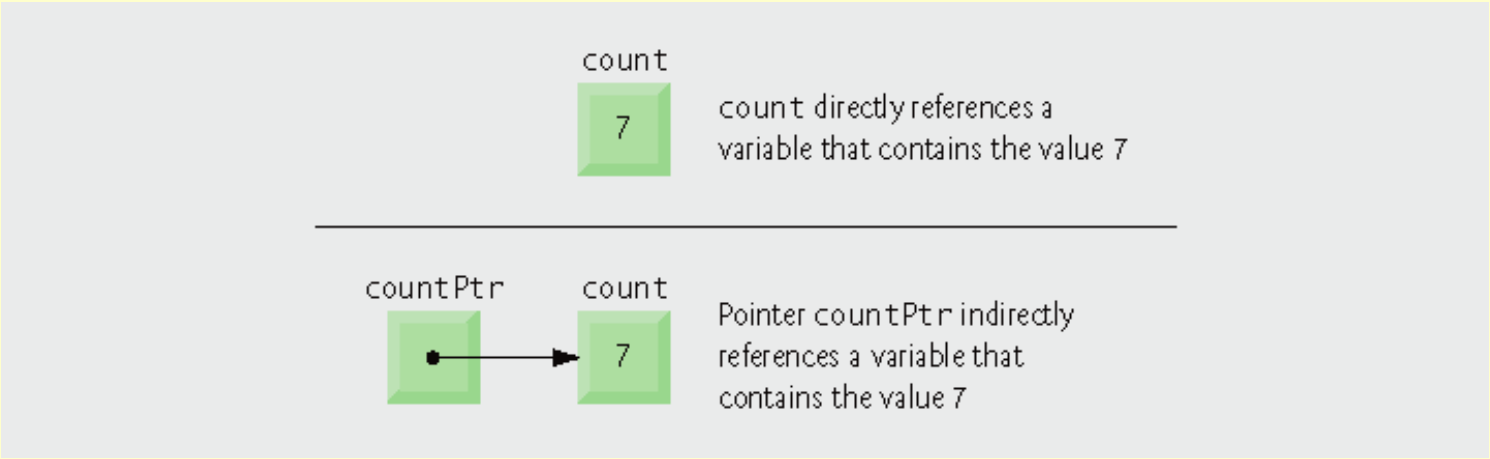
8.2 Pointer Variable Declarations and Initialization (Cont.)

- Pointer declarations
  - * indicates variable is a pointer
    - Example
      - `int *myPtr;`
        - Declares pointer to `int`, of type `int *`.
        - Note that the name of the pointer variable is `myPtr` but not `*myPtr`.
    - Multiple pointers require multiple asterisks.

```
int *myPtr1, *myPtr2;
```

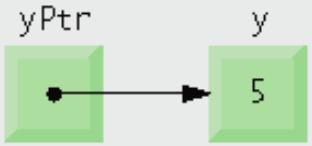
8.2 Pointer Variable Declarations and Initialization (Cont.)

- Pointer initialization
  - Initialized to 0, NULL, or an address
    - 0 or NULL points to nothing (null pointer).



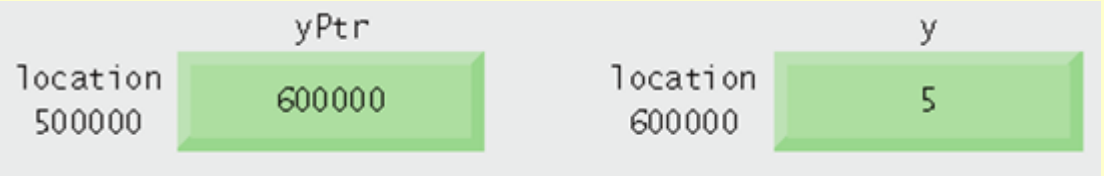
8.3 Pointer Operators

- Address operator (&)
  - Returns memory address of its operand.
  - Example
    - `int y = 5;`
      `int *yPtr;`
      `yPtr = &y;`
      assigns the address of variable `y` to pointer variable `yPtr`.
      - Variable `yPtr` "points to" `y`.
        - `yPtr` indirectly references variable y's value.



8.3 Pointer Operators (Cont.)

- * operator
  - Also called indirection operator or dereferencing operator
  - Returns synonym for the object its operand points to.
  - *yPtr returns



  - Dereferenced pointer is an *lvalue.*

```
*yPtr = 5;
```

8.3 Pointer Operators (Cont.)

- * and & are inverses of each other.
  - Will “cancel one another out” when applied consecutively in either order.

```
1  // Fig. 8.4: fig08_04.cpp
2  // Using the & and * operators.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  int main()
8  {
9     int a; // a is an integer
10    int *aPtr; // aPtr is an int * -- pointer to an integer
11
12    a = 7; // assigned 7 to a
13    aPtr = &a; // assign the address of a to aPtr
```

Outline

fig08_04.cpp

(1 of 2)

```
14
15    cout << "The address of a is " << &a
16       << "\nThe value of aPtr is " << aPtr;
17    cout << "\n\nThe value of a is " << a
18       << "\nThe value of *aPtr is " << *aPtr;
19    cout << "\n\nShowing that * and & are inverses of "
20       << "each other.\n&*aPtr = " << &*aPtr
21       << "\n*&aPtr = " << *&aPtr << endl;
22    return 0; // indicates successful termination
23 } // end main
```

```
The address of a is 0012F580
The value of aPtr is 0012F580

The value of a is 7
The value of *aPtr is 7

Showing that * and & are inverses of each other.
&*aPtr = 0012F580
*&aPtr = 0012F580
```

Outline

fig08_04.cpp

(2 of 2)

A Complete Example

- int a = 7; int *aPtr; aPtr = &a;

Location (address)	Variable Name	Value
2000	aPtr	0012F580
0012F580	a	7

  - aPtr = &a = 0012F580
  - *aPtr = a = 7
  - &aPtr = 2000

A Complete Example

- &*aPtr = &(*aPtr) = &a = 0012F580
- *&aPtr = *(&aPtr) = *(2000) = 0012F580
- *aPtr = 9 $\Rightarrow$ (automatically) a = 9
- a = 9 $\Rightarrow$ (automatically) *aPtr = 9

Another Example

- Two variables are declared below:

```
float b = 2.5;
float *bPtr;
bPtr = &b;
```

- The output of the program is listed below:

```
b = 2.5

&b = 0012FF60

&bPtr = 0012FF54
```

Another Example

- Questions
  - What is the name of the pointer variable? bPtr
  - What is the memory location of the variable b? 0012FF60
  - What is the value stored in the variable b? 2.5
  - What is the value of *(&b)? 2.5
  - What is the value of &(*bPtr)? 0012FF60
  - What is the value of &(*b)? Error

8.4 Passing Arguments to Functions by Reference with Pointers

- Three ways to pass arguments to a function
  - Pass-by-value
  - Pass-by-reference with reference arguments
  - Pass-by-reference with pointer arguments

8.4 Passing Arguments to Functions by Reference with Pointers

- Arguments passed to a function using reference arguments
  - Function can modify original values of arguments.
  - Advantage: More than one value “returned”

8.4 Passing Arguments to Functions by Reference with Pointers (Cont.)

- Pass-by-reference with pointer arguments
  - Simulates pass-by-reference.
    - Use pointers and indirection operator
  - Passes address of argument using & operator.
  - Arrays are not passed with & because array name is already a pointer.
  - * operator is used as alias/nickname for variable inside of function.

Outline

fig08_06.cpp

(1 of 1)

```
1  // Fig. 8.6: fig08_06.cpp
2  // Cube a variable using pass-by-value.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  int cubeByValue( int ); // prototype
8
9  int main()
10 {
11    int number = 5;
12
13    cout << "The original value of number is " << number;
14
15    number = cubeByValue( number ); // pass number by value to cubeByValue
16    cout << "\nThe new value of number is " << number << endl;
17    return 0; // indicates successful termination
18 } // end main
19
20 // calculate and return cube of integer argument
21 int cubeByValue( int n )
22 {
23    return n * n * n; // cube local variable n and return result
24 } // end function cubeByValue
```

```
The original value of number is 5
The new value of number is 125
```

Outline

fig08_07.cpp

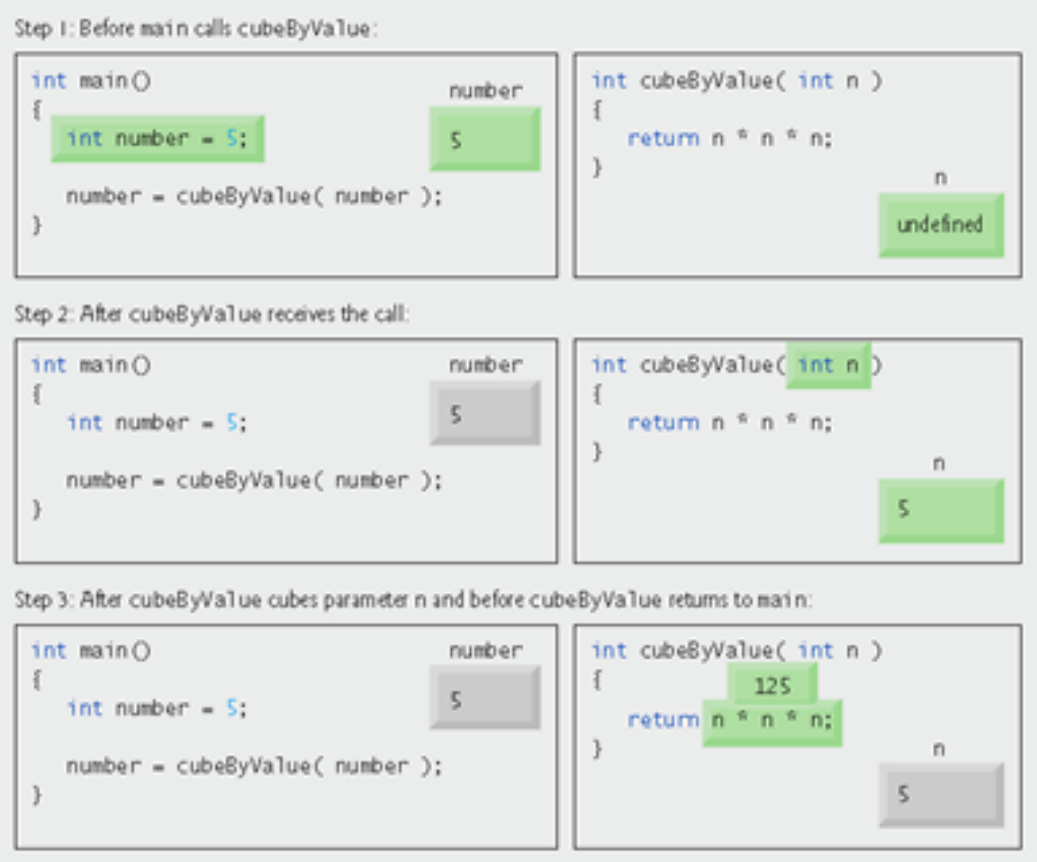
(1 of 1)

```
1  // Fig. 8.7: fig08_07.cpp
2  // Cube a variable using pass-by-reference with a pointer argument.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  void cubeByReference( int * ); // prototype
8
9  int main()
10 {
11    int number = 5;
12
13    cout << "The original value of number is " << number;
14
15    cubeByReference( &number ); // pass number address to cubeByReference
16
17    cout << "\nThe new value of number is " << number << endl;
18    return 0; // indicates successful termination
19 } // end main
20
21 // calculate cube of *nPtr; modifies variable number in main
22 void cubeByReference( int *nPtr )
23 {
24    *nPtr = *nPtr * *nPtr * *nPtr; // cube *nPtr
25 } // end function cubeByReference
```

```
The original value of number is 5
The new value of number is 125
```

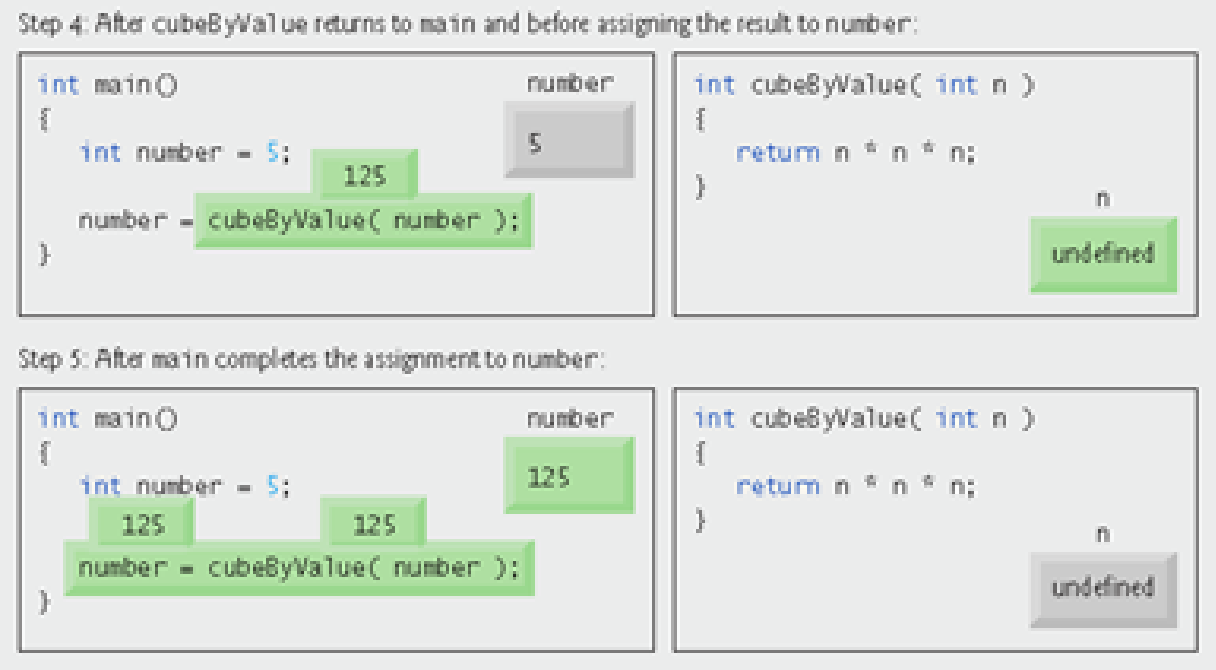
8.4 Passing Arguments to Functions by Reference with Pointers (Cont.)

- Pass-by-value analysis



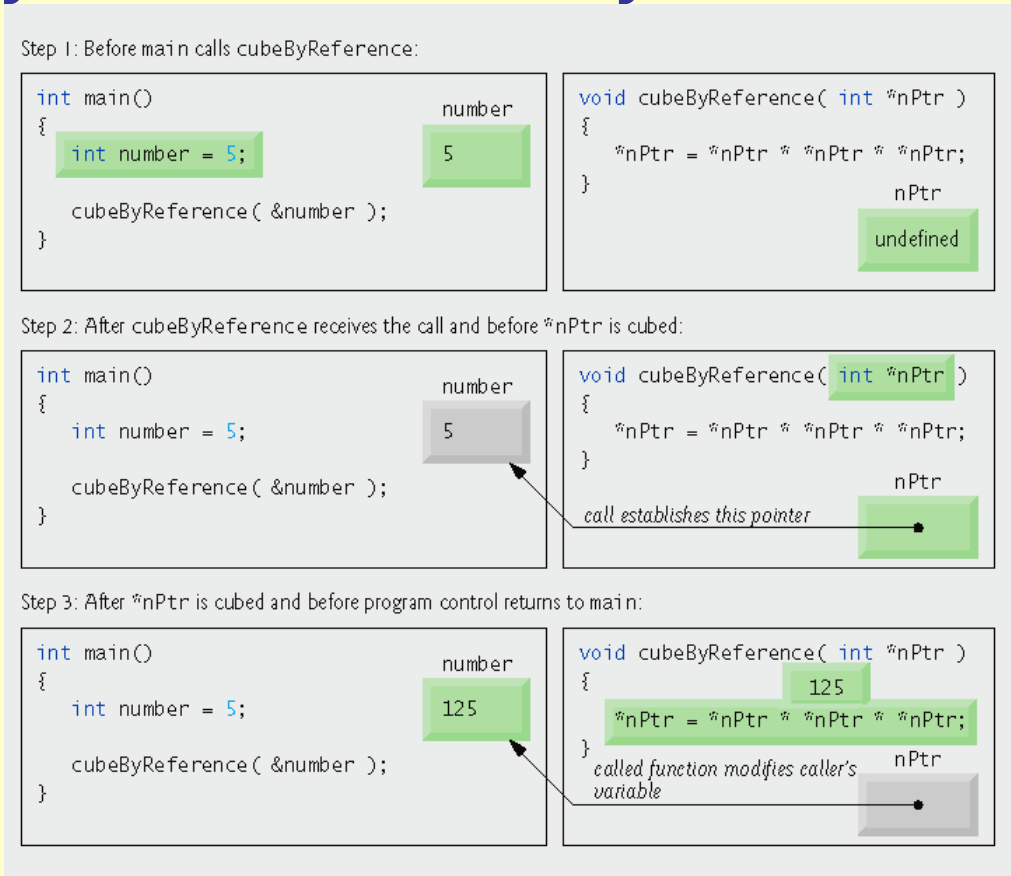
8.4 Passing Arguments to Functions by Reference with Pointers (Cont.)

- Pass-by-value analysis (cont.)



8.4 Passing Arguments to Functions by Reference with Pointers (Cont.)

- Pass-by-reference analysis



8.7 `sizeof` Operators

- `sizeof` operator
  - Returns size of operand in bytes.
  - For arrays, `sizeof` returns

    (size of 1 element) * (number of elements)
  - If `sizeof( int )` returns 4 then

```
int myArray[ 10 ];
cout << sizeof( myArray );
```

    will print 40.
  - Can be used with
    - Variable names
    - Type names
    - Constant values

Outline

fig08_16.cpp

(1 of 1)

```
1  // Fig. 8.16: fig08_16.cpp
2  // Sizeof operator when used on an array name
3  // returns the number of bytes in the array.
4  #include <iostream>
5  using std::cout;
6  using std::endl;
7
8  size_t getSize( double * ); // prototype
9
10 int main()
11 {
12    double array[ 20 ]; // 20 doubles; occupies 160 bytes on our system
13
14    cout << "The number of bytes in the array is " << sizeof( array );
15
16    cout << "\nThe number of bytes returned by getSize is "
17       << getSize( array ) << endl;
18    return 0; // indicates successful termination
19 } // end main
20
21 // return size of ptr
22 size_t getSize( double *ptr )
23 {
24    return sizeof( ptr );
25 } // end function getSize
```

```
The number of bytes in the array is 160
The number of bytes returned by getSize is 4
```

```
1  // Fig. 8.17: fig08_17.cpp
2  // Demonstrating the sizeof operator.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  int main()
8  {
9     char c; // variable of type char
10    short s; // variable of type short
11    int i; // variable of type int
12    long l; // variable of type long
13    float f; // variable of type float
14    double d; // variable of type double
15    long double ld; // variable of type long double
16    int array[ 20 ]; // array of int
17    int *ptr = array; // variable of type int *
```

Outline

fig08_17.cpp

(1 of 2)

Outline

fig08_17.cpp

(2 of 2)

```
18
19    cout << "sizeof c = " << sizeof c
20       << "\tsizeof(char) = " << sizeof( char )
21       << "\nsizeof s = " << sizeof s
22       << "\tsizeof(short) = " << sizeof( short )
23       << "\nsizeof i = " << sizeof i
24       << "\tsizeof(int) = " << sizeof( int )
25       << "\nsizeof l = " << sizeof l
26       << "\tsizeof(long) = " << sizeof( long )
27       << "\nsizeof f = " << sizeof f
28       << "\tsizeof(float) = " << sizeof( float )
29       << "\nsizeof d = " << sizeof d
30       << "\tsizeof(double) = " << sizeof( double )
31       << "\nsizeof ld = " << sizeof ld
32       << "\tsizeof(long double) = " << sizeof( long double )
33       << "\nsizeof array = " << sizeof array
34       << "\nsizeof ptr = " << sizeof ptr << endl;
35    return 0; // indicates successful termination
36 } // end main
```

```
sizeof c = 1    sizeof(char) = 1
sizeof s = 2    sizeof(short) = 2
sizeof i = 4    sizeof(int) = 4
sizeof l = 4    sizeof(long) = 4
sizeof f = 4    sizeof(float) = 4
sizeof d = 8    sizeof(double) = 8
sizeof ld = 8   sizeof(long double) = 8
sizeof array = 80
sizeof ptr = 4
```

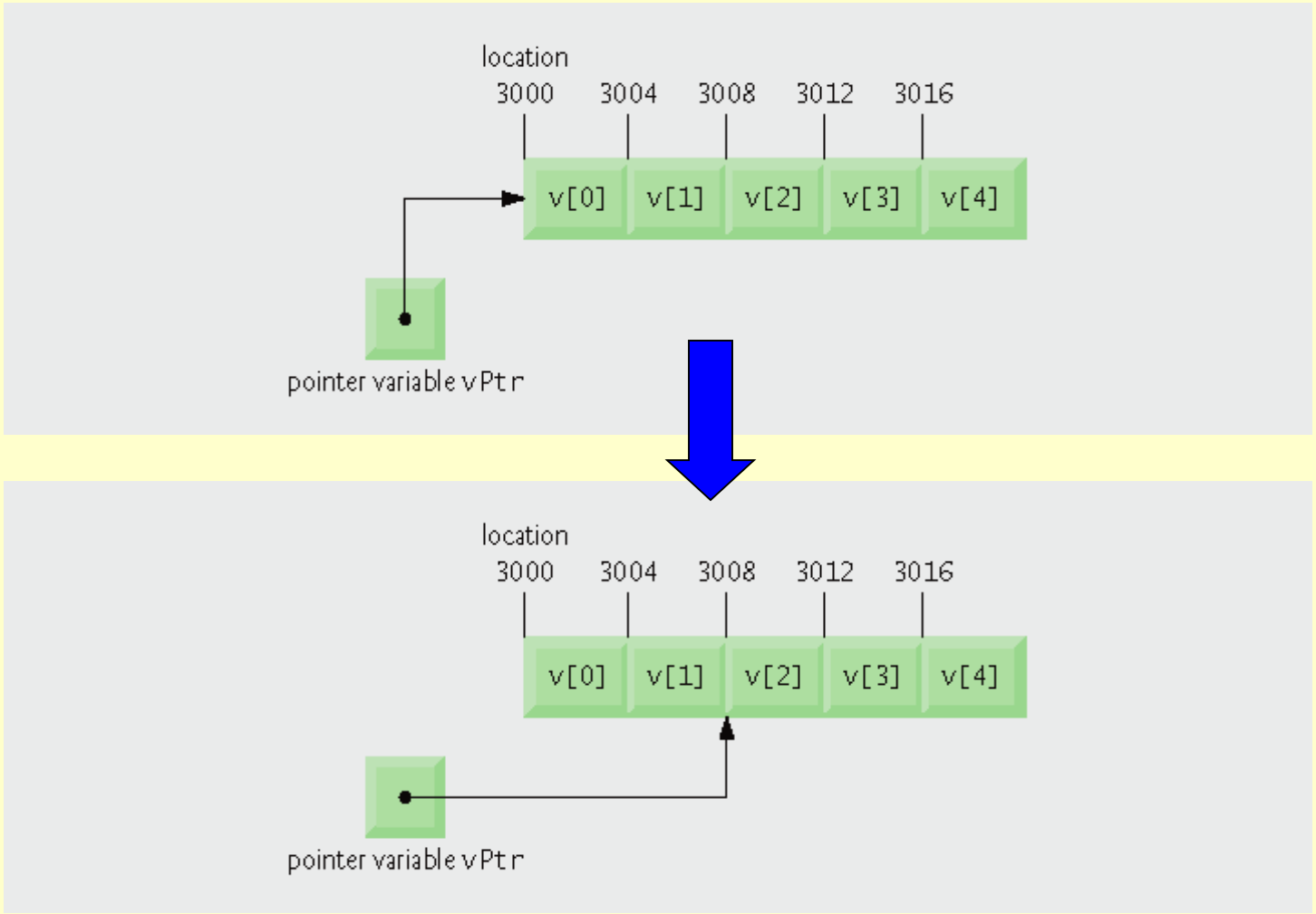
8.8 Pointer Expressions and Pointer Arithmetic

- Pointer arithmetic
  - Increment/decrement pointer (++ or −−).
  - Add/subtract an integer to/from a pointer (+ or +=, − or −=).
  - Pointers may be subtracted from each other.
  - Pointer arithmetic is meaningless unless performed on a pointer to an array.

8.8 Pointer Expressions and Pointer Arithmetic (Cont.)

- 5 element `int` array on a machine using 4 byte `int`s
  - `vPtr = &v[ 0 ];`
  - `vPtr` points to first element `v[ 0 ]`, at location 3000.
  - `vPtr += 2;`
  - Set `vPtr` to 3008 (`3000 + 2 * 4`). `vPtr` points to `v[ 2 ]`.

8.8 Pointer Expressions and Pointer Arithmetic (Cont.)



8.8 Pointer Expressions and Pointer Arithmetic (Cont.)

- Subtracting pointers
  - Returns number of elements between two addresses.

```
vPtr2 = v[ 2 ];
vPtr = v[ 0 ];
vPtr2 - vPtr is 2
```

8.8 Pointer Expressions and Pointer Arithmetic (Cont.)

- Pointer assignment
  - Pointer can be assigned to another pointer if both are of same type.
    - If not same type, cast operator must be used.
    - Exception
      - Pointer to `void` (type `void *`)
        - Generic pointer, represents any type
        - No casting needed to convert pointer to `void *`
        - Casting is needed to convert `void *` to any other type.

8.8 Pointer Expressions and Pointer Arithmetic (Cont.)

- Pointer comparison
  - Use equality and relational operators.
  - Compare addresses stored in pointers.
    - Comparisons are meaningless unless pointers point to members of the same array.
  - Example
    - Could show that one pointer points to higher-index element of array than another pointer.
  - Commonly used to determine whether pointer is 0 (null pointer).

8.9 Relationship Between Pointers and Arrays

- Arrays and pointers are closely related
  - Array name is like constant pointer.
  - Pointers can do array subscripting operations.

8.9 Relationship Between Pointers and Arrays (Cont.)

- Accessing array elements with pointers
  - Assume declarations:

```
int b[ 5 ];
int *bPtr;
bPtr = b;
```

  - Element `b[ n ]` can be accessed by `*( bPtr + n )`
    - Called pointer/offset notation

8.9 Relationship Between Pointers and Arrays (Cont.)

- Addresses
  - `&b[ 3 ]` is same as `bPtr + 3`
- Array name can be treated as pointer
  - `b[ 3 ]` is same as `*( b + 3 )`
- Pointers can be subscripted (pointer/subscript notation)
  - `bPtr[ 3 ]` is same as `b[ 3 ]`

Outline

fig08_20.cpp

(1 of 3)

```
1  // Fig. 8.20: fig08_20.cpp
2  // Using subscripting and pointer notations with arrays.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  int main()
8  {
9     int b[] = { 10, 20, 30, 40 }; // create 4-element array b
10    int *bPtr = b; // set bPtr to point to array b
11
12    // output array b using array subscript notation
13    cout << "Array b printed with:\n\nArray subscript notation\n";
14
15    for ( int i = 0; i < 4; i++ )
16       cout << "b[" << i << "] = " << b[ i ] << '\n';
17
18    // output array b using the array name and pointer/offset notation
19    cout << "\nPointer/offset notation where "
20       << "the pointer is the array name\n";
21
22    for ( int offset1 = 0; offset1 < 4; offset1++ )
23       cout << "*(b + " << offset1 << ") = " << *( b + offset1 ) << '\n';
```

```
24
25    // output array b using bPtr and array subscript notation
26    cout << "\nPointer subscript notation\n";
27
28    for ( int j = 0; j < 4; j++ )
29       cout << "bPtr[" << j << "] = " << bPtr[ j ] << '\n';
30
31    cout << "\nPointer/offset notation\n";
32
33    // output array b using bPtr and pointer/offset notation
34    for ( int offset2 = 0; offset2 < 4; offset2++ )
35       cout << "*(bPtr + " << offset2 << ") = "
36          << *( bPtr + offset2 ) << '\n';
37
38    return 0; // indicates successful termination
39 } // end main
```

Outline

fig08_20.cpp

(2 of 3)

```
Array b printed with:

Array subscript notation
b[0] = 10
b[1] = 20
b[2] = 30
b[3] = 40

Pointer/offset notation where the pointer is the array name
*(b + 0) = 10
*(b + 1) = 20
*(b + 2) = 30
*(b + 3) = 40

Pointer subscript notation
bPtr[0] = 10
bPtr[1] = 20
bPtr[2] = 30
bPtr[3] = 40

Pointer/offset notation
*(bPtr + 0) = 10
*(bPtr + 1) = 20
*(bPtr + 2) = 30
*(bPtr + 3) = 40
```

Outline

fig08_20.cpp

(3 of 3)

Outline

fig08_21.cpp

(1 of 2)

```
1  // Fig. 8.21: fig08_21.cpp
2  // Copying a string using array notation and pointer notation.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  void copy1( char *, const char * ); // prototype
8  void copy2( char *, const char * ); // prototype
9
10 int main()
11 {
12    char string1[ 10 ];
13    const char *string2 = "Hello";
14    char string3[ 10 ];
15    char string4[] = "Good Bye";
16
17    copy1( string1, string2 ); // copy string2 into string1
18    cout << "string1 = " << string1 << endl;
19
20    copy2( string3, string4 ); // copy string4 into string3
21    cout << "string3 = " << string3 << endl;
22    return 0; // indicates successful termination
23 } // end main
```

```
24
25 // copy s2 to s1 using array notation
26 void copy1( char * s1, const char * s2 )
27 {
28    // copying occurs in the for header
29    for ( int i = 0; ( s1[ i ] = s2[ i ] ) != '\0'; i++ )
30       ; // do nothing in body
31 } // end function copy1
32
33 // copy s2 to s1 using pointer notation
34 void copy2( char *s1, const char *s2 )
35 {
36    // copying occurs in the for header
37    for ( ; ( *s1 = *s2 ) != '\0'; s1++, s2++ )
38       ; // do nothing in body
39 } // end function copy2
```

```
string1 = Hello
string3 = Good Bye
```

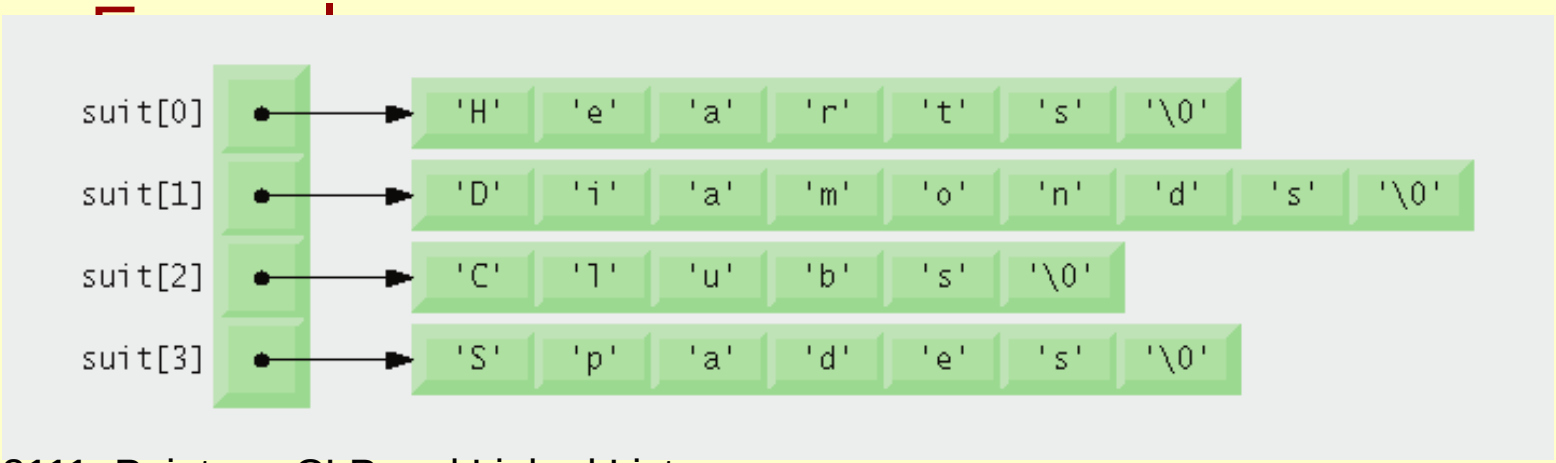
Outline

fig08_21.cpp

(2 of 2)

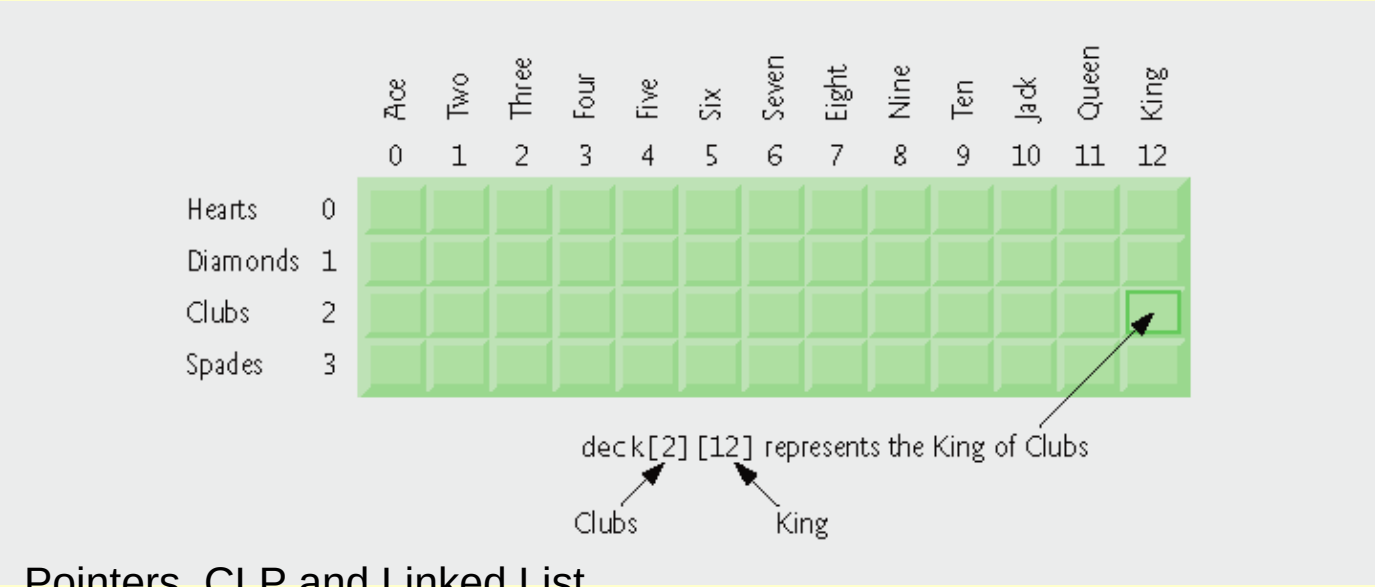
8.10 Arrays of Pointers

- Arrays can contain pointers.
  - Commonly used to store array of strings (string array)
  - Array does not store strings, only pointers to strings.



8.11 Case Study: Card Shuffling and Dealing Simulation

- Card shuffling program
  - Use an array of pointers to strings, to store suit names.
  - Use a double scripted array (suit-by-value).
  -



8.13.1 Fundamentals of Characters and Pointer-Based Strings

- Character constant
  - Integer value represented as character in single quotes
    - Example
      - `'z'` is integer value of z
        - 122 in ASCII
      - `'\n'` is integer value of newline
        - 10 in ASCII

8.13.1 Fundamentals of Characters and Pointer-Based Strings (Cont.)

- String
  - Series of characters treated as single unit
  - Can include letters, digits, special characters +, -, *, ...
  - Array of characters, ends with null character '\0'
  - String is constant pointer.
    - Pointer to string's first character
      - Like arrays

8.13.1 Fundamentals of Characters and Pointer-Based Strings (Cont.)

- String assignment
  - Character array
    - `char color[] = "blue";`
      - Creates 5 element `char` array `color`
        - Last element is `'\0'`.
  - Variable of type `char *`
    - `const char *colorPtr = "blue";`
      - Creates pointer `colorPtr` to letter `b` in string `"blue"`
  - Alternative for character array
    - `char color[] = { 'b', 'l', 'u', 'e', '\0' };`

8.13.1 Fundamentals of Characters and Pointer-Based Strings (Cont.)

- Reading strings
  - Assign input to character array `word[ 20 ]`.
    - `cin >> word;`
      - Reads characters until whitespace or EOF.
  - String could exceed array size
    - `cin >> setw( 20 ) >> word;`
      - Reads only up to 19 characters (space reserved for `'\0'`).

8.13.1 Fundamentals of Characters and Pointer-Based Strings (Cont.)

- `cin.getline`
  - Read line of text
    - `cin.getline( array, size, delimiter );`
      - Copies input into specified `array` until either
        - One less than `size` is reached
        - `delimiter` character is input
    - Example
      - ```
        char sentence[ 80 ];
        cin.getline( sentence, 80, '\n' );
        ```
        ```

8.13.2 String Manipulation Functions of the String-Handling Library

- Copying strings
  - `char *strcpy_s( char *s1, const char *s2 )`
    - Copies second argument into first argument.
      - First argument must be large enough to store string and terminating null character.
  - `char *strncpy_s( char *s1, const char *s2, size_t n )`
    - Specifies number of characters to be copied from second argument into first argument.
      - A null character will be added at the end of `s1`.

Outline

fig08_31.cpp

(1 of 2)

```
1  // Fig. 8.31: fig08_31.cpp
2  // Using strcpy and strncpy.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  #include <cstring> // prototypes for strcpy and strncpy
8  using std::strcpy;
9  using std::strncpy;
10
11 int main()
12 {
13    char x[] = "Happy Birthday to You"; // string length 21
14    char y[ 25 ];
15    char z[ 15 ];
16
17    strcpy_s( y, x ); // copy contents of x into y
18
19    cout << "The string in array x is: " << x
20       << "\nThe string in array y is: " << y << '\n';
```

```
21
22    // copy first 14 characters of x into z
23    strncpy_s( z, x, 14 ); // copy null character too
24
25
26    cout << "The string in array z is: " << z << endl;
27    return 0; // indicates successful termination
28 } // end main
```

```
The string in array x is: Happy Birthday to You
The string in array y is: Happy Birthday to You
The string in array z is: Happy Birthday
```

Outline

fig08_31.cpp

(2 of 2)

8.13.2 String Manipulation Functions of the String-Handling Library (Cont.)

- Concatenating strings
  - `char *strcat_s( char *s1, const char *s2 )`
    - Appends second argument to first argument.
      - First character of second argument replaces null character terminating first argument.
      - You must ensure first argument large enough to store concatenated result and null character.
  - `char *strncat_s( char *s1, const char *s2, size_t n )`
    - Appends specified number of characters from second argument to first argument.
      - Appends terminating null character to result.

Outline

fig08_32.cpp

(1 of 2)

```
1  // Fig. 8.32: fig08_32.cpp
2  // Using strcat and strncat.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  #include <cstring> // prototypes for strcat and strncat
8  using std::strcat;
9  using std::strncat;
10
11 int main()
12 {
13    char s1[ 20 ] = "Happy "; // length 6
14    char s2[] = "New Year "; // length 9
15    char s3[ 40 ] = "";
16
17    cout << "s1 = " << s1 << "\ns2 = " << s2;
18
19    strcat_s( s1, s2 ); // concatenate s2 to s1 (length 15)
20
21    cout << "\n\nAfter strcat(s1, s2):\ns1 = " << s1 << "\ns2 = " << s2;
22
23    // concatenate first 6 characters of s1 to s3
24    strncat_s( s3, s1, 6 ); // places '\0' after last character
25
26    cout << "\n\nAfter strncat(s3, s1, 6):\ns1 = " << s1
27       << "\ns3 = " << s3;
```

```
28
29    strcat_s( s3, s1 ); // concatenate s1 to s3
30    cout << "\n\nAfter strcat(s3, s1):\ns1 = " << s1
31       << "\ns3 = " << s3 << endl;
32    return 0; // indicates successful termination
33 } // end main
```

```
s1 = Happy
s2 = New Year

After strcat(s1, s2):
s1 = Happy New Year
s2 = New Year

After strncat(s3, s1, 6):
s1 = Happy New Year
s3 = Happy

After strcat(s3, s1):
s1 = Happy New Year
s3 = Happy Happy New Year
```

Outline

fig08_32.cpp

(2 of 2)

8.13.2 String Manipulation Functions of the String-Handling Library (Cont.)

- Comparing strings
  - `int strcmp( const char *s1, const char *s2 )`
    - Compares character by character.
    - Returns
      - Zero if strings are equal
      - Negative value if first string is less than second string
      - Positive value if first string is greater than second string

8.13.2 String Manipulation Functions of the String-Handling Library (Cont.)

- `int strncmp( const char *s1, const char *s2, size_t n )`
  - Compares up to specified number of characters.
    - Stops if it reaches null character in one of arguments.

Outline

fig08_33.cpp

(1 of 2)

```
1  // Fig. 8.33: fig08_33.cpp
2  // Using strcmp and strncmp.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  #include <iomanip>
8  using std::setw;
9
10 #include <cstring> // prototypes for strcmp and strncmp
11 using std::strcmp;
12 using std::strncmp;
13
14 int main()
15 {
16    const char *ss1 = "Happy New Year";
17    const char *ss2 = "Happy New Year";
18    const char *ss3 = "Happy Holidays";
19
20    cout << "ss1 = " << ss1 << "\nss2 = " << ss2 << "\nss3 = " << ss3
21       << "\n\nstrcmp(ss1, ss2) = " << setw( 2 ) << strcmp( ss1, ss2 )
22       << "\nstrcmp(ss1, ss3) = " << setw( 2 ) << strcmp( ss1, ss3 )
23       << "\nstrcmp(ss3, ss1) = " << setw( 2 ) << strcmp( ss3, ss1 );
24
25    cout << "\n\nstrncmp(ss1, ss3, 6) = " << setw( 2 )
26       << strncmp( ss1, ss3, 6 ) << "\nstrncmp(ss1, ss3, 7) = " << setw( 2 )
27       << strncmp( ss1, ss3, 7 ) << "\nstrncmp(ss3, ss1, 7) = " << setw( 2 )
28       << strncmp( ss3, ss1, 7 ) << endl;
29    return 0; // indicates successful termination
30 } // end main
```

```
ss1 = Happy New Year
ss2 = Happy New Year
ss3 = Happy Holidays

strcmp(ss1, ss2) =  0
strcmp(ss1, ss3) =  1
strcmp(ss3, ss1) = -1

strncmp(ss1, ss3, 6) =  0
strncmp(ss1, ss3, 7) =  1
strncmp(ss3, ss1, 7) = -1
```

Outline

fig08_33.cpp

(2 of 2)

8.13.2 String Manipulation Functions of the String-Handling Library (Cont.)

- Tokenizing (Cont.)
  - `char *strtok( char *s1, const char *s2 )`
    - Multiple calls required
      - First call contains two arguments, string to be tokenized and string containing delimiting characters.
        - Finds next delimiting character and replaces with null character.
      - Subsequent calls continue tokenizing
        - Call with first argument `NULL`
        - Stores pointer to remaining string in a `static` variable.
    - Returns pointer to current token.

Outline

fig08_34.cpp

(1 of 2)

```cpp
// Fig. 8.34: fig08_34.cpp
// Using strtok.
#include <iostream>
using std::cout;
using std::endl;

#include <cstring> // prototype for strtok
using std::strtok;

int main()
{
   char sentence[] = "This is a sentence with 7 tokens";
   char *tokenPtr, *nextToken;
   char separators[] = " ,\t\n";
   cout << "The string to be tokenized is:\n" << sentence
      << "\n\nThe tokens are:\n\n";

   // begin tokenization of sentence
   tokenPtr = strtok_s( sentence, separators, &nextToken );

   // continue tokenizing sentence until tokenPtr becomes NULL
   while ( tokenPtr != NULL )
   {
      cout << tokenPtr << '\n';
      tokenPtr = strtok_s( NULL, separators, &nextToken); // get next token
   } // end while

   cout << "\nAfter strtok, sentence = " << sentence << endl;
   return 0; // indicates successful termination
} // end main
```

```
The string to be tokenized is:
This is a sentence with 7 tokens

The tokens are:

This
is
a
sentence
with
7
tokens

After strtok, sentence = This
```

Outline

fig08_34.cpp

(2 of 2)

8.13.2 String Manipulation Functions of the String-Handling Library (Cont.)

- Determining string lengths
  - `size_t strlen( const char *s )`
    - Returns number of characters in string.
      - Terminating null character is not included in length
      - This length is also the index of the terminating null character.

Outline

fig08_35.cpp

(1 of 1)

```
1  // Fig. 8.35: fig08_35.cpp
2  // Using strlen.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  #include <cstring> // prototype for strlen
8  using std::strlen;
9
10 int main()
11 {
12    const char *sss1 = "abcdefghijklmnopqrstuvwxyz";
13    const char *sss2 = "four";
14    const char *sss3 = "Boston";
15
16    cout << "The length of \"" << sss1 << "\" is " << strlen( sss1 )
17       << "\nThe length of \"" << sss2 << "\" is " << strlen( sss2 )
18       << "\nThe length of \"" << sss3 << "\" is " << strlen( sss3 )
19       << endl;
20    return 0; // indicates successful termination
21 } // end main
```

```
The length of "abcdefghijklmnopqrstuvwxyz" is 26
The length of "four" is 4
The length of "Boston" is 6
```

Command-Line Processing

- On many systems, it is possible to pass arguments to main from a command line.

```
int main(int argc, char *argv[])
```

  - **`argc`** receives the number of command-line arguments.
  - **`argv`** is an array of **`char *`**'s pointing to strings in which the actual command-line arguments are stored.

Command-Line Processing

- All arguments passing in are strings.
- We may use the following functions to covert them into other types.
  - atoi(str): convert a string str into an integer
  - atol(str): convert a string str into a long integer
  - atof(str): convert a string str into a floating point number.

Command-Line Processing

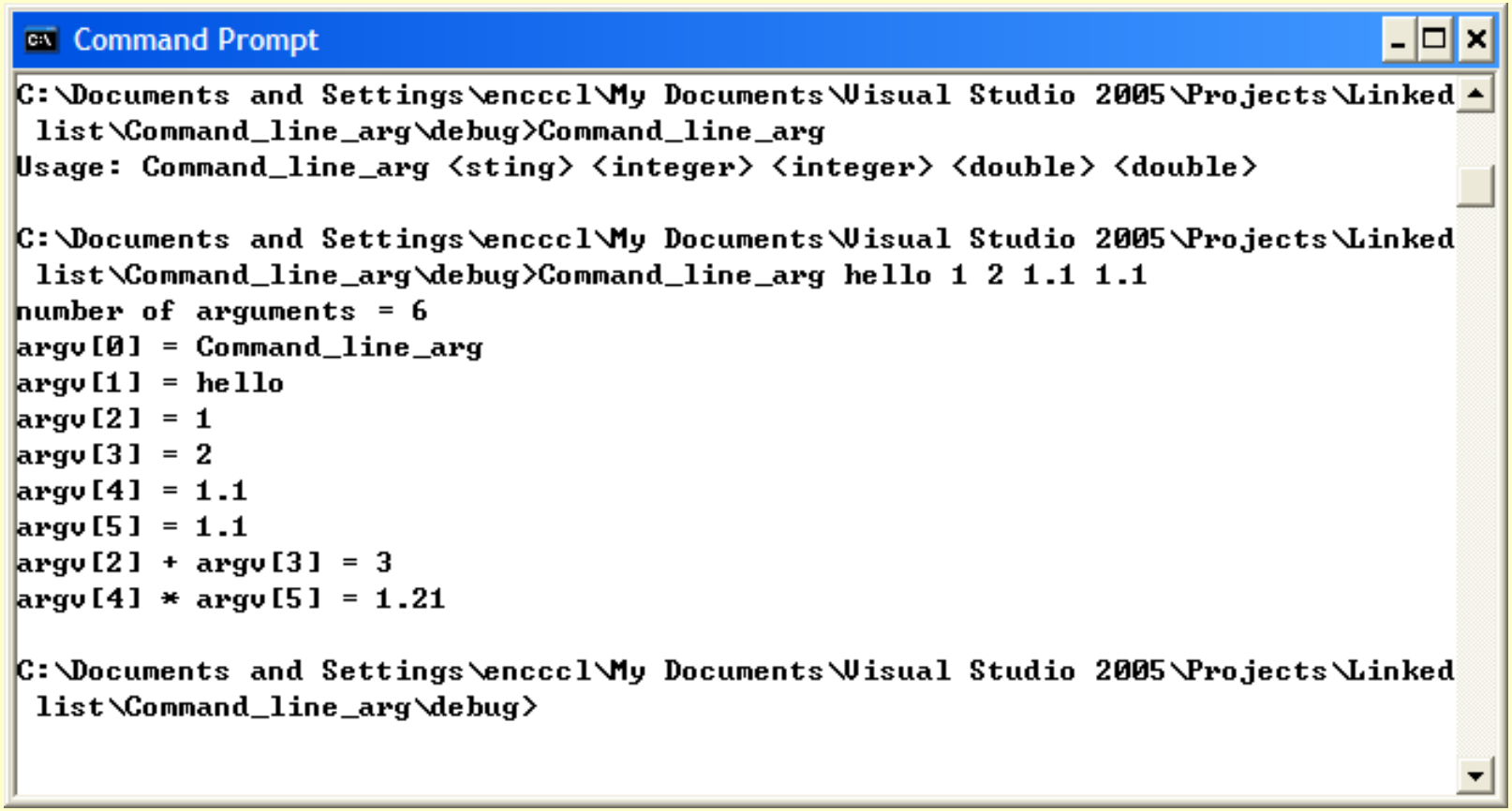
- Example

```
int main(int argc, char *argv[])
{
    int i1, i2; // integers to store the 3rd and 4th arguments
    double d1, d2; // floating points to store 5th and 6th arguments

    //check number of command-line arguments
    if (argc != 6)
        cout << "Usage: Command_line_arg <sting> <integer> <integer> <double>
    <double>" << endl;
```

Command-Line Processing

```
    else
    {
            // display each argument
            cout << "number of arguments = " << argc << endl;

            for (int i = 0; i < argc; i++)
                    cout << "argv[" << i << "] = " << argv[i] << endl;
            // integer calculation
            i1 = atoi(argv[2]);
            i2 = atoi(argv[3]);
            cout << "argv[2] + argv[3] = " << i1 + i2 << endl;

            // floating point calcuation
            d1 = atof(argv[4]);
            d2 = atof(argv[5]);
            cout << "argv[4] * argv[5] = " << d1 * d2 << endl;
    }
    return 0;
}
```

Command-Line Processing

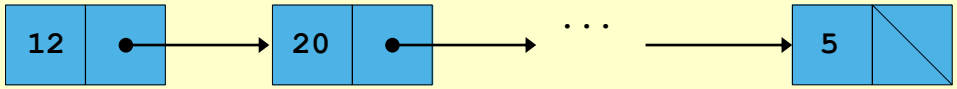


Linked List

- In using array, one needs to define the size beforehand.
- If the size is too big, it is a waste of memory.
- If the size is too small, it may even crash the system.
- Solution: linked list

Linked List

- Linked list is a data structure that consists of a number items, with each item having a pointer points to the location of the next item.



Linked List

- Declaration

```
class Singly_linked_list // Use a class Singly_linked_list to represent an object
{
    public:
    // constructor initialize the nextPtr
    Singly_linked_list()
    {
        nextPtr = 0; // point to null at the beginning
    }

    // get a number
    int GetNum()
    {
        return number;
    }
```

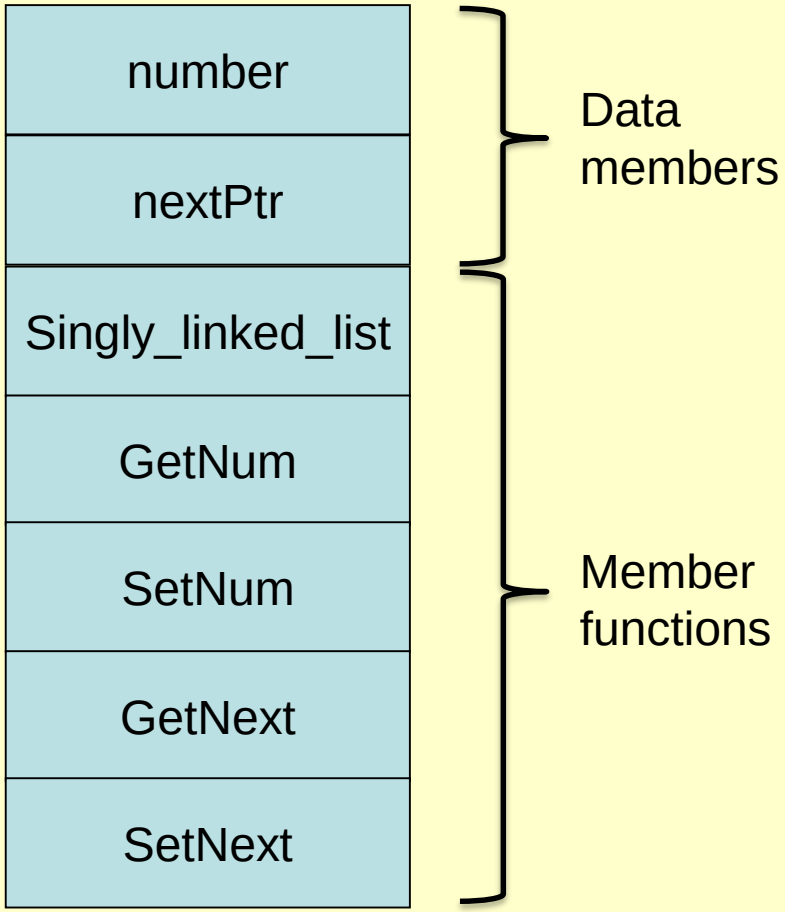
Linked List

```
// set a number
void SetNum(int num)
{
        number = num;
}

// get the next pointer
Singly_linked_list *GetNext()
{
        return nextPtr;
}
```

Linked List

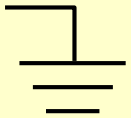
```
    // set the next pointer
    void SetNext(Singly_linked_list *ptr)
    {
            nextPtr = ptr;
    }

private:
    int number;
    Singly_linked_list *nextPtr;
};
```

An object of the class `Singly_linked_list`



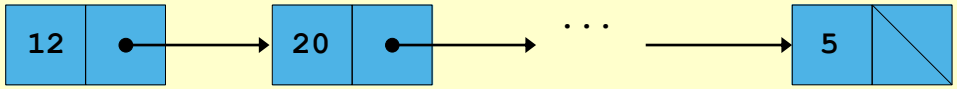
Linked List

- Collection of self-referential class objects (nodes) connected by pointers (links)
  - Accessed using pointer to first node of list
    - Subsequent nodes accessed using the links in each node
  - Link in last node is null (zero)
    - Indicates end of list
  - Data stored dynamically
    - Nodes created as necessary
    - Node can have data of any type

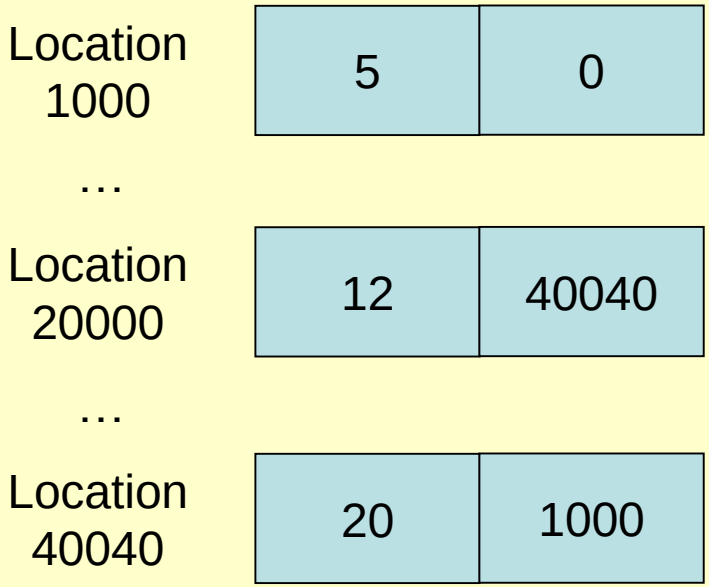

or


Linked List

- A graphical representation of a list



- The list in the computer memory



Linked List

- Linked lists vs. arrays
  - Arrays can become full
    - Allocating "extra" space in array wasteful, may never be used.
    - Linked lists can grow/shrink as needed.
    - Linked lists only become full when system runs out of memory.
  - Linked lists can be maintained in sorted order
    - Insert element at proper position.
    - Existing elements do not need to be moved.

Linked List

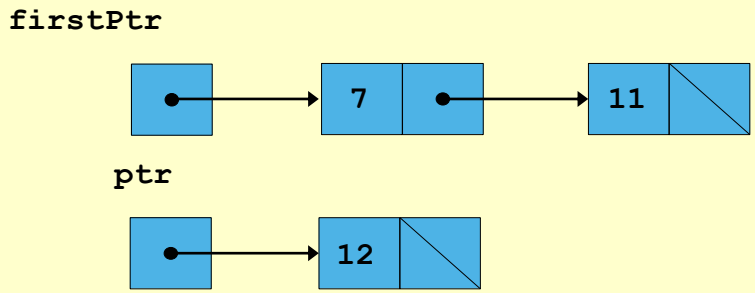
- Selected linked list operations
  - Insert node at front
  - Insert node at back
  - Insert a node in the list after searching
  - Remove node from front
  - Remove node from back
  - Remove a node from the list after searching

Linked List

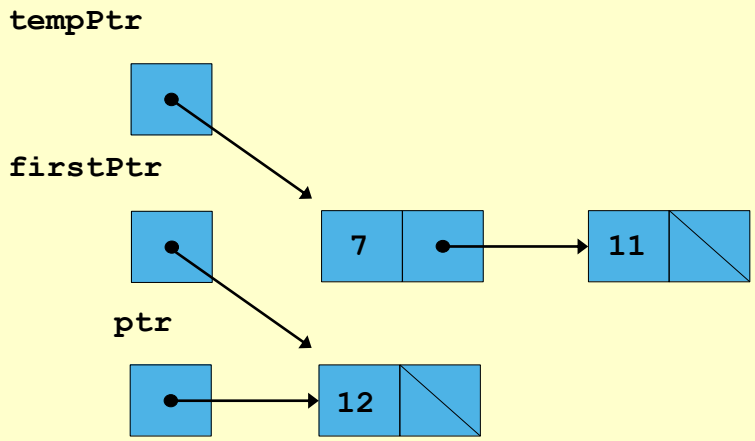
- In following illustrations
  - **`firstPtr`** points to the first element (object) of the linked list.
  - **`ptr`** points to the object that inserts to the linked list or removes from the linked list.
  - **`delete`** is a function to delete an object.

Linked List: Insert at front

Step 1:

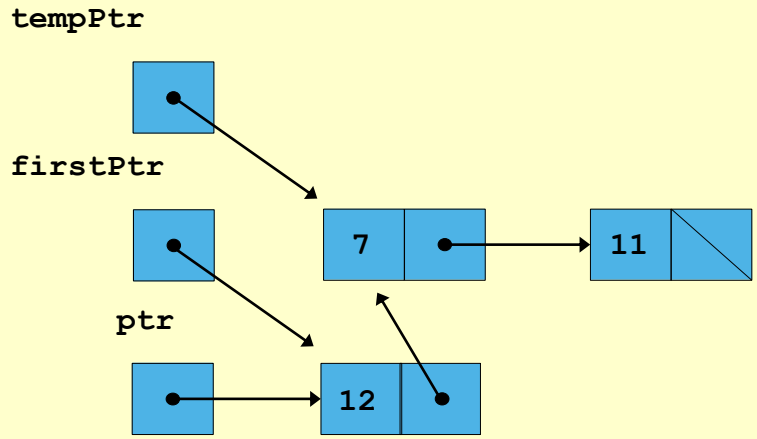


Step 2:

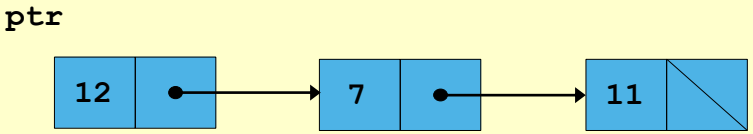


Linked List: Insert at front

Step 3:



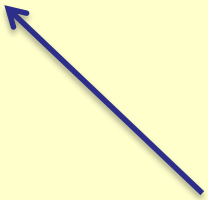
Final:



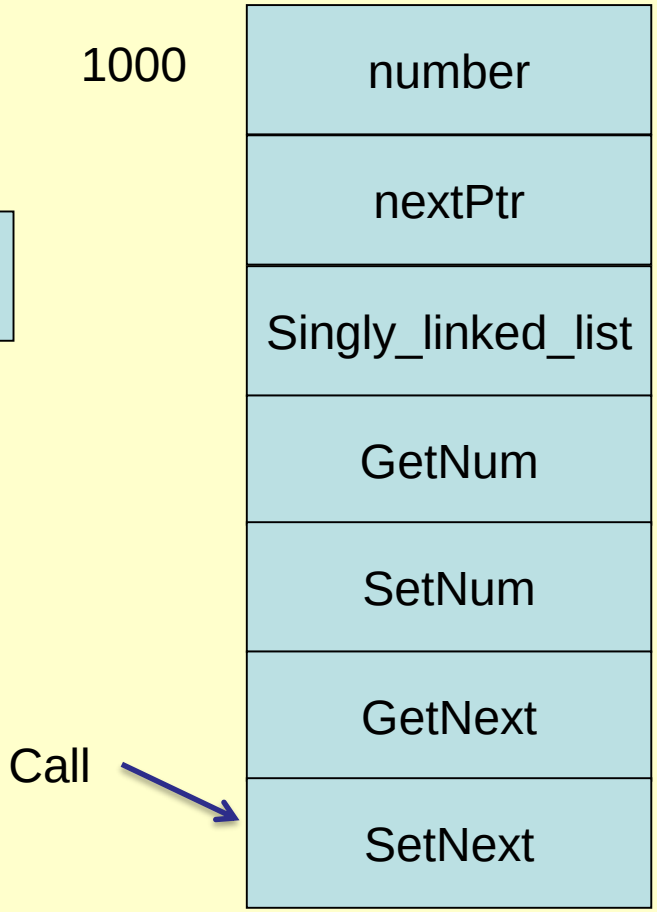
`ptr` points to the first element of the new linked list

Linked List: Insert at front

```
// insert a node at the front
Singly_linked_list *Insert_node_at_front(Singly_linked_list *ptr, Singly_linked_list *firstPtr)
{
    Singly_linked_list *tempPtr = 0;

    tempPtr = firstPtr;
    firstPtr = ptr;
    ptr->SetNext(tempPtr);
    return ptr;
}
```

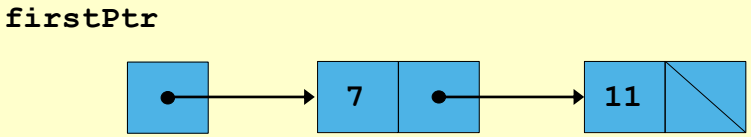


"->": Call the member function ***SetNext*** of the object pointed by the pointer ***ptr***.

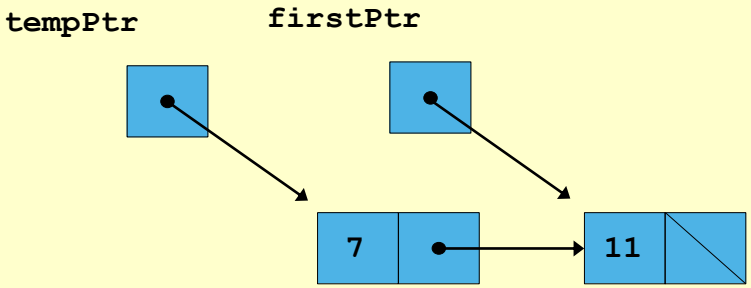


Linked List: Remove from front

Step 1:

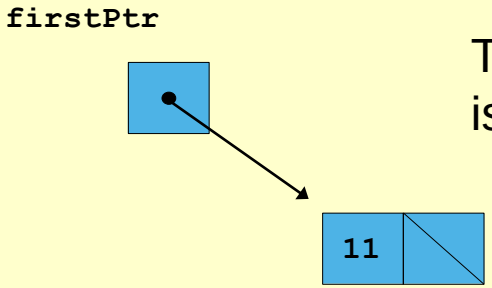


Step 2:



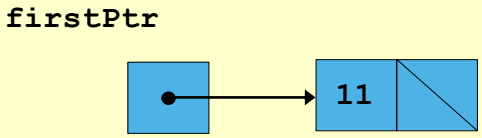
Linked List: Remove from front

Step 3:



The object pointed by `tempPtr` is deleted.

Final:



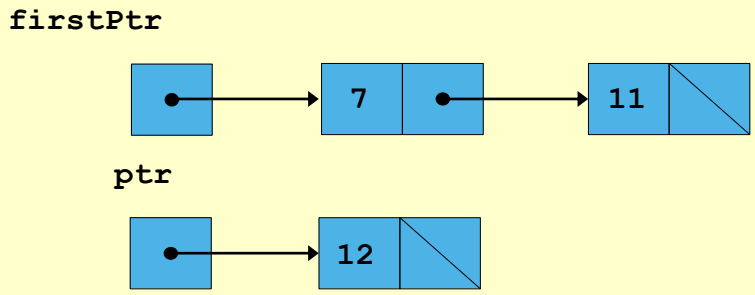
`firstPtr` points to the first element of the new linked list

Linked List: Remove from front

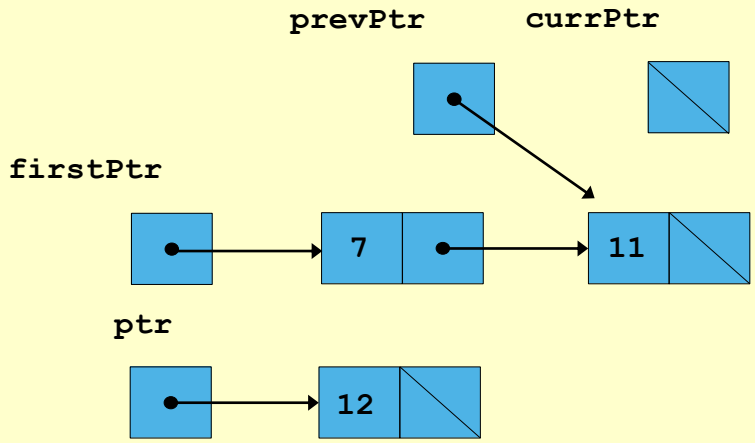
```
// remove a node at the front
Singly_linked_list *Remove_node_at_front(Singly_linked_list *firstPtr)
{
    Singly_linked_list *tempPtr = 0;

    tempPtr = firstPtr;
    firstPtr = firstPtr->GetNext();
    delete tempPtr;
    return firstPtr;
}
```

Linked List: Insert at back

Step 1:

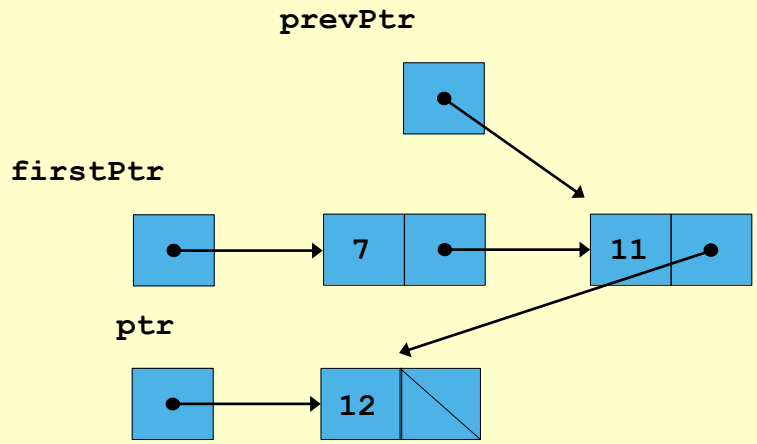


Step 2:

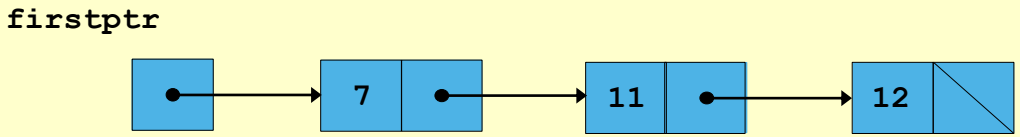


Linked List: Insert at back

Step 3:



Final:



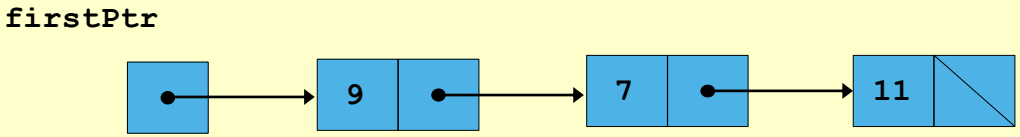
`firstPtr` points to the first element of the new linked list

Linked List: Insert at back

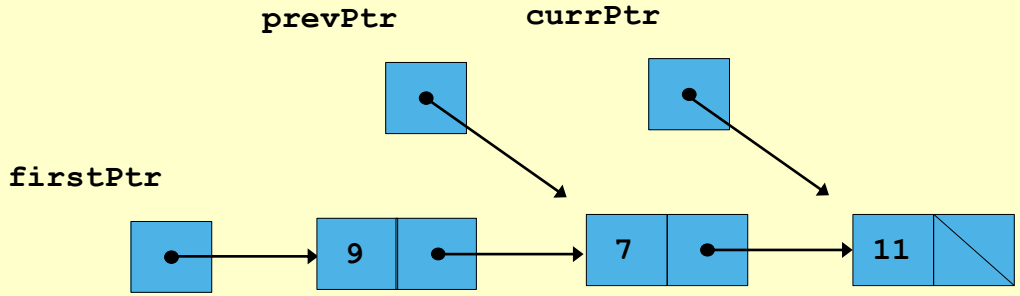
```
// insert a node at the back
Singly_linked_list *Insert_node_at_back(Singly_linked_list *ptr, Singly_linked_list *firstPtr)
{
    Singly_linked_list *prevPtr = 0, *currPtr = 0;

    currPtr = firstPtr;
    while (currPtr != 0)
    {
        prevPtr = currPtr;
        currPtr = currPtr->GetNext();
    }
    prevPtr->SetNext(ptr); // prevPtr = lastPtr
    return firstPtr;
}
```

Linked List: Remove from back

Step 1:

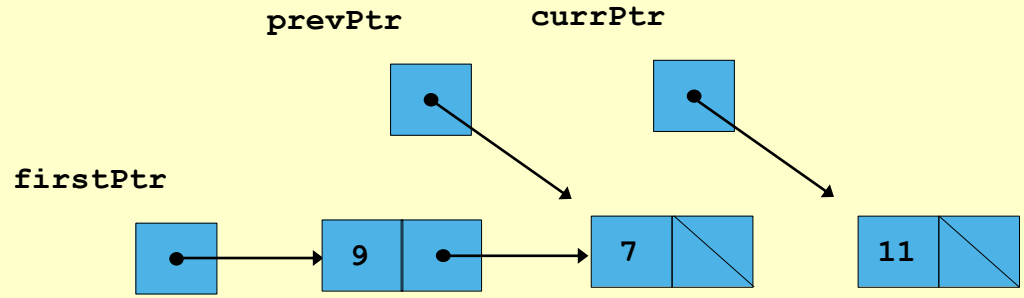


Step 2:

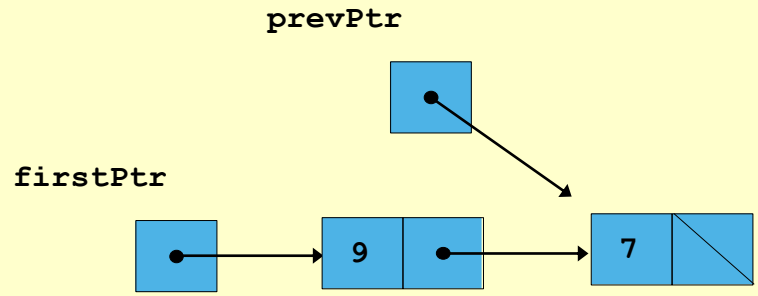


Linked List: Remove from back

Step 3:



Final:



The object pointed by `currPtr` is deleted.

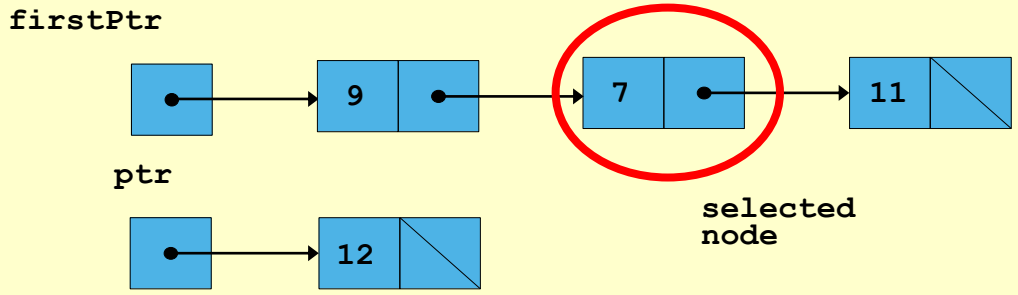
`firstPtr` points to the first element of the new linked list

Linked List: Remove from back

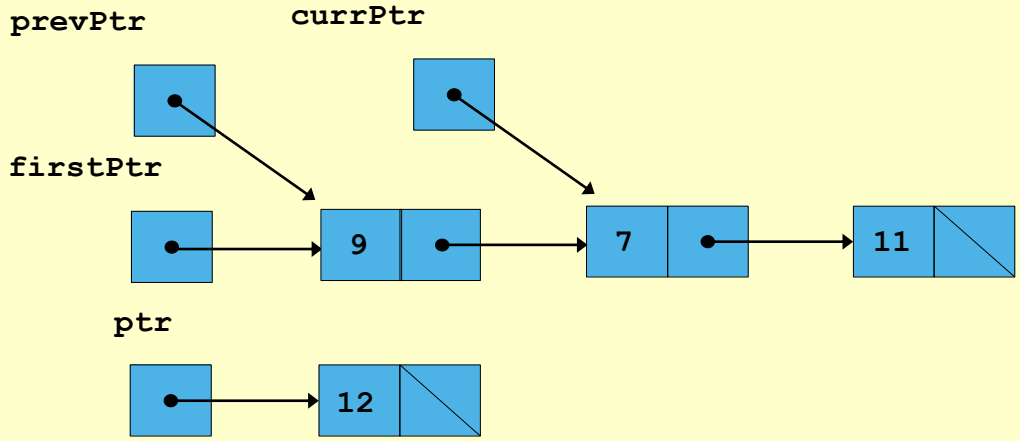
```
// remove a node at the back
Singly_linked_list *Remove_node_at_back(Singly_linked_list *firstPtr)
{
    Singly_linked_list *prevPtr = 0, *currPtr = 0;

    currPtr = firstPtr;
    while(currPtr->GetNext() != 0)
    {
        prevPtr = currPtr;
        currPtr = currPtr->GetNext();
    }
    prevPtr->SetNext(0);
    delete currPtr;
    return firstPtr;
}
```

Linked List: Selected Insert

Step 1:

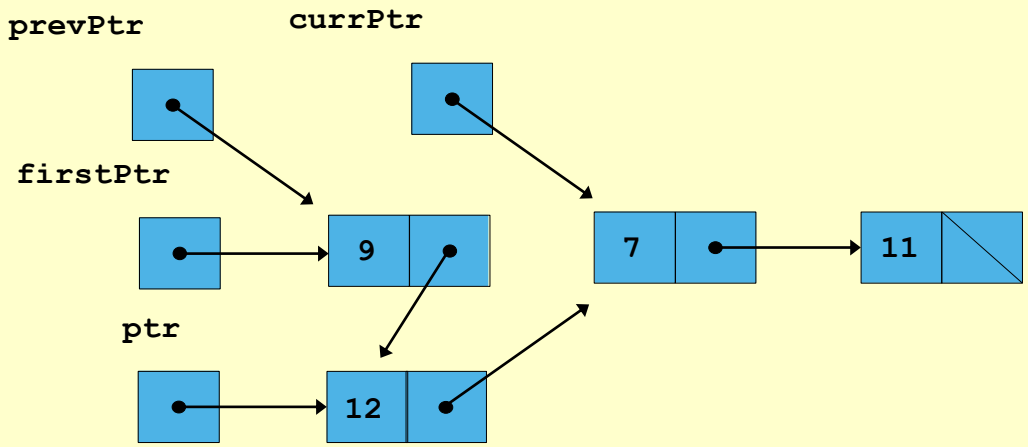


Step 2:

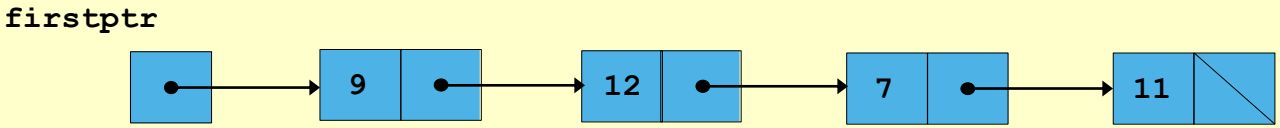


Linked List: Selected Insert

Step 3:



Final:



`firstPtr` points to the first element of the new linked list

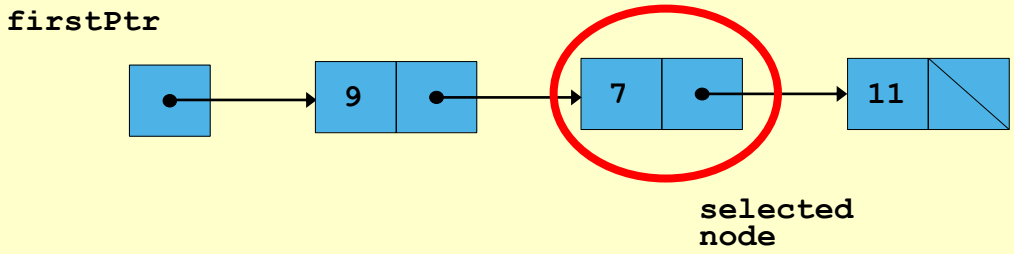
Linked List: Selected Insert

- Insert a node before the node with a given value in the list

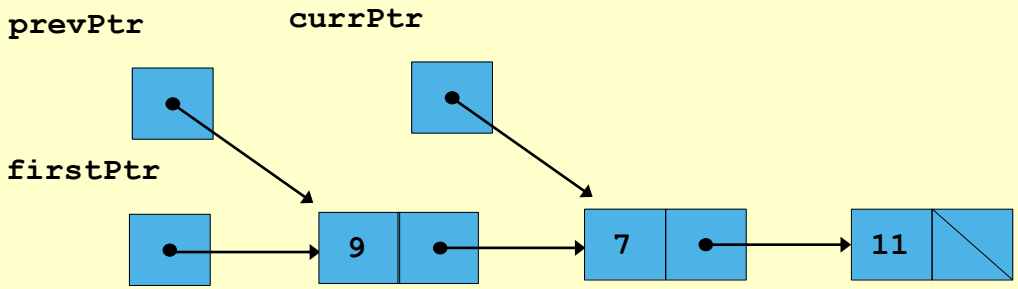
```
// insert a node before the node with the given value
Singly_linked_list *Insert_node(int n, Singly_linked_list *ptr, Singly_linked_list *firstPtr) {
    Singly_linked_list *prevPtr = 0, *currPtr = 0;
    currPtr = firstPtr;
    while (currPtr->GetNum() != n) {
        prevPtr = currPtr;
        currPtr = currPtr->GetNext();
    }
    prevPtr->SetNext(ptr);
    ptr->SetNext(currPtr);
    return firstPtr;
}
```

Linked List: Selected Remove

Step 1:

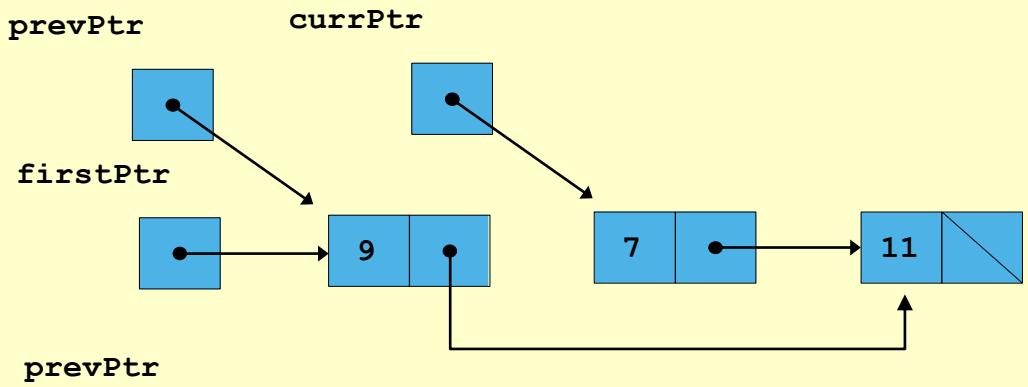


Step 2:

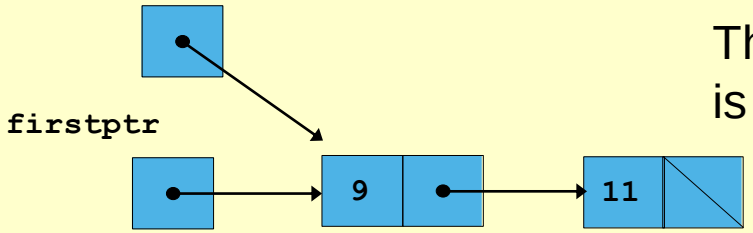


Linked List: Selected Remove

Step 3:



Final:



The object pointed by `currPtr` is deleted.

`firstPtr` points to the first element of the new linked list

Linked List: Selected Remove

- Remove a node with a given value from the list

```
// remove a node with the given value
Singly_linked_list *Remove_node(int n, Singly_linked_list *firstPtr) {
    Singly_linked_list *prevPtr = 0, *currPtr = 0;

    currPtr = firstPtr;
    while(currPtr->GetNum() != n) {
        prevPtr = currPtr;
        currPtr = currPtr->GetNext();
    }
    prevPtr->SetNext(currPtr->GetNext());
    delete currPtr;
    return firstPtr;
}
```

Linked List

- The complete program

```
#include<iostream>
using std::cin;
using std::cout;
using std::endl;

class Singly_linked_list // Use a class Singly_linked_list to represent an object
{
	public:
	// constructor initialize the nextPtr
	Singly_linked_list()
	{
		nextPtr = 0; // point to null at the beginning
	}

	// get a number
	int GetNum()
	{
		return number;
	}
```

Linked List

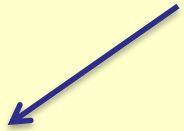
```
    // set a number
    void SetNum(int num)
    {
            number = num;
    }

    // get the next pointer
    Singly_linked_list *GetNext()
    {
            return nextPtr;
    }

    // set the next pointer
    void SetNext(Singly_linked_list *ptr)
    {
            nextPtr = ptr;
    }

private:
    int       number;
    Singly_linked_list   *nextPtr;
};
```

Linked List

```
// print a singly linked list
void Print_Singly_linked_list(Singly_linked_list *ptr)
{
	while (ptr != 0)
	{
		cout << "[" << ptr->GetNum() << "]->";
		ptr = ptr->GetNext();
	}
	cout << "Null" << endl;
	cout << endl;
}

// create a singly linked list
Singly_linked_list *Create_Singly_linked_list(int n)
{
	Singly_linked_list  *tempPtr = 0, *firstPtr = 0, *lastPtr = 0;
	int	i = 0;
```

Linked List

"new": create a new object from the class Singly_linked_list and the object is pointed by the pointer tempPtr. Note that the object has no name (variable name).



```
    do
    {
            tempPtr = new Singly_linked_list;
            tempPtr->SetNum(i);
            if (i == 0)
            {
                    firstPtr = tempPtr;
                    lastPtr = tempPtr;
                    i++;
            }
            else
            {
                    lastPtr->SetNext(tempPtr);
                    lastPtr = tempPtr;
                    i++;
            }
    }while (i < n);
    return firstPtr;
}
```

Linked List

```
// insert a node at the front
Singly_linked_list *Insert_node_at_front(Singly_linked_list *ptr, Singly_linked_list *firstPtr)
{
	Singly_linked_list *tempPtr = 0;

	tempPtr = firstPtr;
	firstPtr = ptr;
	ptr->SetNext(tempPtr);
	return ptr;
}

// remove a node at the front
Singly_linked_list *Remove_node_at_front(Singly_linked_list *firstPtr)
{
	Singly_linked_list *tempPtr = 0;

	tempPtr = firstPtr;
	firstPtr = firstPtr->GetNext();
	delete tempPtr;
	return firstPtr;
}
```

Linked List

```
// insert a node at the back
Singly_linked_list *Insert_node_at_back(Singly_linked_list *ptr, Singly_linked_list *firstPtr)
{
	Singly_linked_list *prevPtr = 0, *currPtr = 0;

	currPtr = firstPtr;
	while (currPtr != 0)
	{
		prevPtr = currPtr;
		currPtr = currPtr->GetNext();
	}
	prevPtr->SetNext(ptr);
	return firstPtr;
}
```

Linked List

```
// remove a node at the back
Singly_linked_list *Remove_node_at_back(Singly_linked_list *firstPtr)
{
	Singly_linked_list *prevPtr = 0, *currPtr = 0;

	currPtr = firstPtr;
	while(currPtr->GetNext() != 0)
	{
		prevPtr = currPtr;
		currPtr = currPtr->GetNext();
	}
	prevPtr->SetNext(0);
	delete currPtr;
	return firstPtr;
}
```

Linked List

```
// insert a node before the node with the given value
Singly_linked_list *Insert_node(int n, Singly_linked_list *ptr, Singly_linked_list *firstPtr)
{
	Singly_linked_list *prevPtr = 0, *currPtr = 0;

	currPtr = firstPtr;
	while (currPtr->GetNum() != n)
	{
		prevPtr = currPtr;
		currPtr = currPtr->GetNext();
	}
	prevPtr->SetNext(ptr);
	ptr->SetNext(currPtr);
	return firstPtr;
}
```

Linked List

```
// remove a node with the given value
Singly_linked_list *Remove_node(int n, Singly_linked_list *firstPtr)
{
	Singly_linked_list *prevPtr = 0, *currPtr = 0;

	currPtr = firstPtr;
	while(currPtr->GetNum() != n)
	{
		prevPtr = currPtr;
		currPtr = currPtr->GetNext();
	}
	prevPtr->SetNext(currPtr->GetNext());
	delete currPtr;
	return firstPtr;
}
```

Linked List

```
int main()
{
	int num;
	Singly_linked_list  *headPtr = 0;
	Singly_linked_list  *newPtr = 0;

	cout << "How many items you want to create? ";
	cin >> num;

	headPtr = Create_Singly_linked_list(num);
	Print_Singly_linked_list(headPtr);

	newPtr = new Singly_linked_list;
	newPtr->SetNum(num);
	cout << "Insert a node at the front of the list:" << endl;
	headPtr = Insert_node_at_front(newPtr, headPtr);
	Print_Singly_linked_list(headPtr);

	cout << "Remove a node from the front of the list:" << endl;
	headPtr = Remove_node_at_front(headPtr);
	Print_Singly_linked_list(headPtr);
```

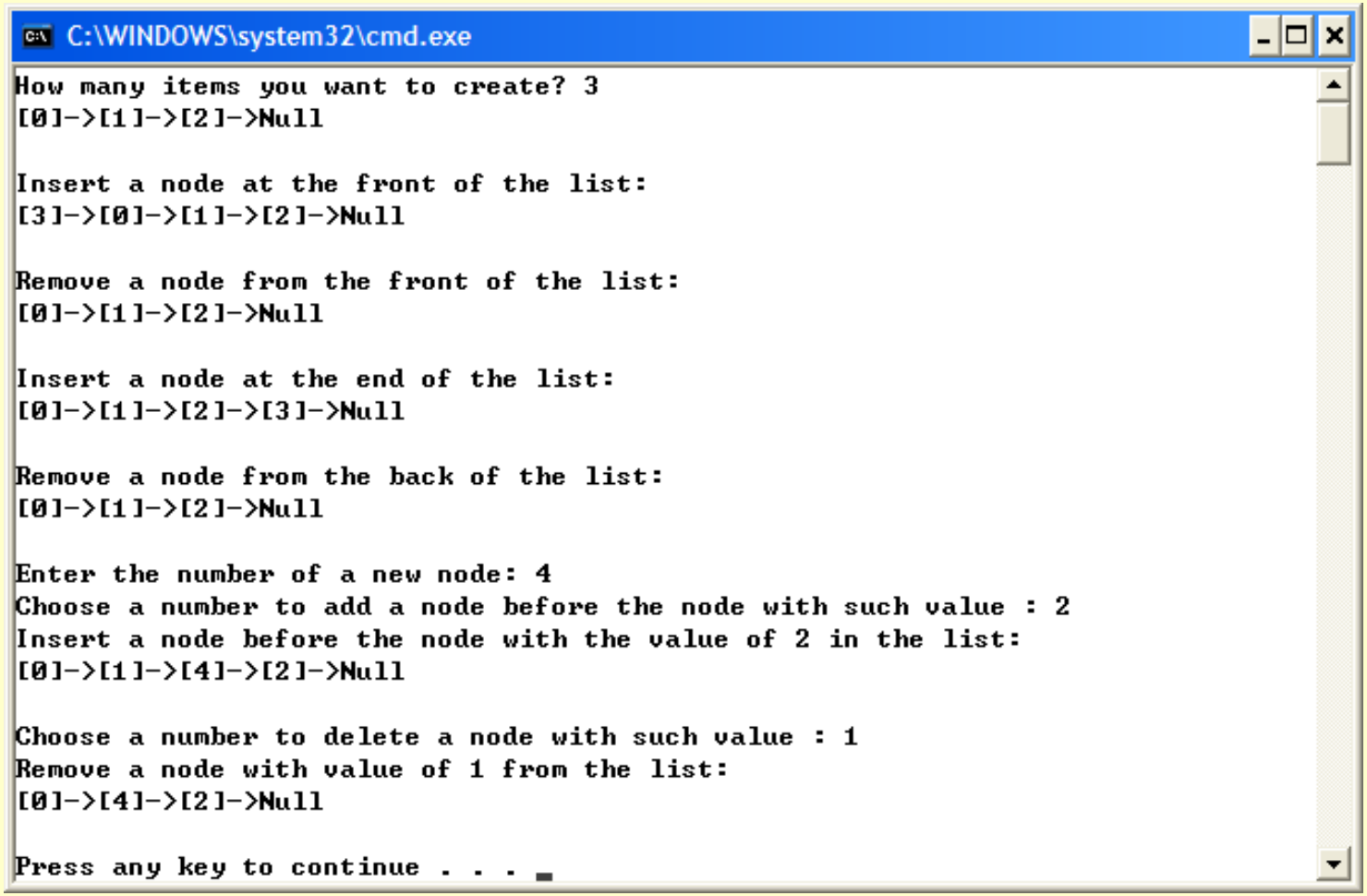
Linked List

```
newPtr = new Singly_linked_list;
newPtr->SetNum(num);
cout << "Insert a node at the end of the list:" << endl;
headPtr = Insert_node_at_back(newPtr, headPtr);
Print_Singly_linked_list(headPtr);

cout << "Remove a node from the back of the list:" << endl;
headPtr = Remove_node_at_back(headPtr);
Print_Singly_linked_list(headPtr);

cout << "Enter the number of a new node: ";
cin >> num;
newPtr = new Singly_linked_list;
newPtr->SetNum(num);
        cout << "Choose a number to add a node before the node with such value : ";
cin >> num;
cout << "Insert a node before the node with the value " << num << " in the list:" << endl;
headPtr = Insert_node(num, newPtr, headPtr);
Print_Singly_linked_list(headPtr);
```

Linked List

```
    cout << "Choose a number to delete a node with such value : ";
    cin >> num;
    cout << "Remove a node with value " << num << " from the list:" << endl;
    headPtr = Remove_node(num, headPtr);
    Print_Singly_linked_list(headPtr);

    return 0;
}
```

Linked List


C:\WINDOWS\system32\cmd.exe
How many items you want to create? 3
[0]->[1]->[2]->Null
Insert a node at the front of the list:
[3]->[0]->[1]->[2]->Null
Remove a node from the front of the list:
[0]->[1]->[2]->Null
Insert a node at the end of the list:
[0]->[1]->[2]->[3]->Null
Remove a node from the back of the list:
[0]->[1]->[2]->Null
Enter the number of a new node: 4
Choose a number to add a node before the node with such value : 2
Insert a node before the node with the value of 2 in the list:
[0]->[1]->[4]->[2]->Null
Choose a number to delete a node with such value : 1
Remove a node with value of 1 from the list:
[0]->[4]->[2]->Null
Press any key to continue . . .

Linked List

- Types of linked lists
  - Singly linked list (used in example)
    - Pointer to first node
    - Travel in one direction (null-terminated)
  - Circular, singly-linked
    - As above, but last node points to first

Linked List

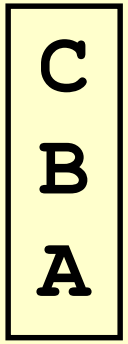
- Doubly linked list
  - Each node has a forward and backwards pointer
  - Travel forward or backward
  - Last node null-terminated
- Circular, double-linked
  - As above, but first and last node joined

Stack

- Nodes can be added/removed from top
  - Constrained version of linked list
  - Like a stack of plates
  - Last-in, first-out (LIFO) data structure
  - Bottom of stack has null link
- Stack operations
  - Push: add node to top
  - Pop: remove node from top
    - Stores value in reference variable

Stack

- Stack applications
  - Function calls: know how to return to caller
    - Return address pushed on stack
    - Most recent function call on top
    - If function A calls B which calls C:



Queue

- Like waiting in line
- Nodes added to back (*tail*), removed from front (*head*)
  - First-in, first-out (FIFO) data structure
  - Insert/remove called enqueue/dequeue

Queue

- Applications
  - Print spooling
    - Documents wait in queue until printer available
  - Packets on network
  - File requests from server

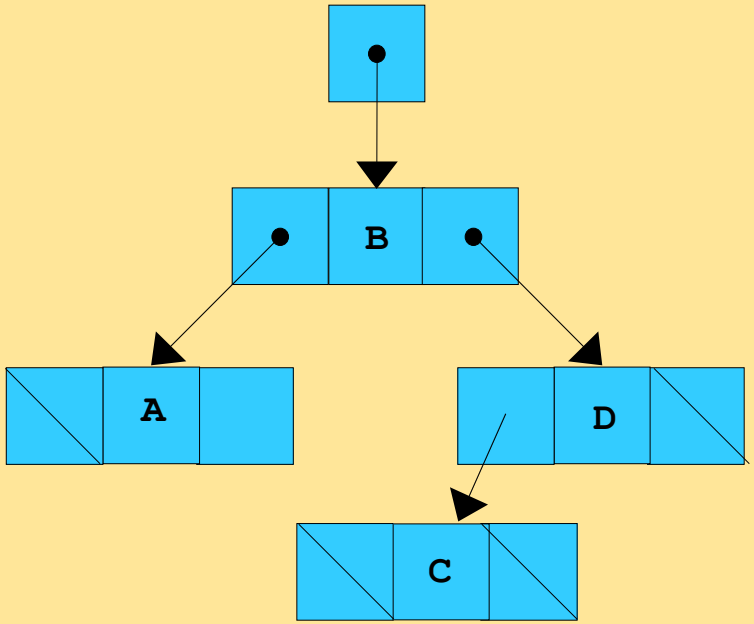
Tree

- Nonlinear, two-dimensional
- Tree nodes have 2 or more links
  - Binary trees have exactly 2 links/node
    - None, both, or one link can be null

Tree

- Terminology
  - *Root node*: first node on tree
  - Link refers to *child* of node
    - Left child is root of *left subtree*
    - Right child is root of *right subtree*
  - *Leaf node*: node with no children
  - Trees drawn from root downwards

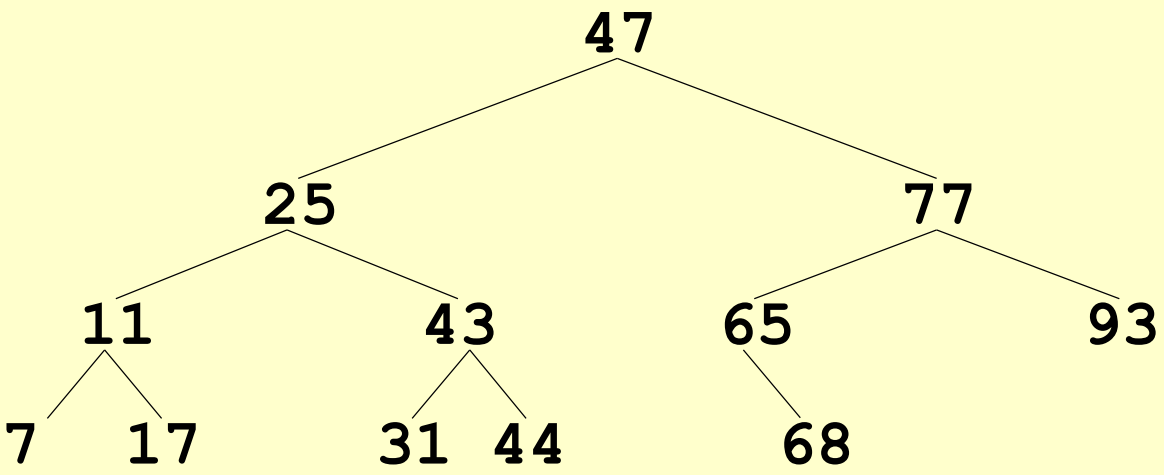
Tree



Tree

- Binary search tree
  - Values in left subtree less than parent node
  - Values in right subtree greater than parent
    - Does not allow duplicate values (good way to remove them)
  - Fast searches, $\log_2 n$ comparisons for a balanced tree

Tree



Until Next Time

- Review Chapter 8 and 21
- Preview Chapter 17